

Bosque del autómatas — Especificación de mecánicas (para el portado a Unity)

Fuente de verdad: el código actual de `pixel_char/world/src/` (rama `claude/atleta-lowpoly-unity-22xtbw`). Todas las constantes de este documento se han leído del código o se han exportado del juego en marcha (`port/tools/export_port_data.mjs` → `port/data/*.json`). Donde la documentación antigua (comentarios, `PROGRESS.md`) dice otra cosa, manda el código; las diferencias encontradas están en el §17.

Cómo leerlo: cada sistema tiene sus **reglas** (lo que debe pasar), sus **números** (con unidades) y, en los algoritmos difíciles, **pseudocódigo**. Lo que es solo presentación (partículas, sonido, brillo del ojo, textos) aparece marcado como **[visual]**: en Unity serán eventos de VFX/SFX/UI, y no cambian la lógica.

0. Convenciones

Cosa	Valor
Unidad de distancia	u = 1 baldosa del mapa. El personaje mide <code>CHAR_H = 1.7 u</code> de pie
«alturas»	múltiplos de <code>CHAR_H</code> (algunas constantes de movimiento están en alturas: $\times 1.7$ para pasar a <code>u</code>)
Plano del suelo	(x, z) ; altura <code>y</code> . Rumbo (<code>heading</code>) = <code>atan2(dz, dx)</code> en radianes, en el suelo
Tiempo	segundos (s) salvo que diga ms
Reloj de juego	<code>t</code> (avanza con el paso; lo usan cámara, partículas)
Reloj de COMBATE	<code>ct</code> (W.ct): avanza con el paso pero se congela en el hitstop y va al 30 % en la cámara lenta. Todas las ventanas de combate (<code>parry</code> , <code>búfer</code> , <code>riposte</code> , <code>carga...</code>) se miden en <code>ct</code>
Paso	<code>dt = min(0.05, tiempo real del frame)</code> (<code>a < 20 fps</code> el juego va más lento que el reloj). Las pruebas usan paso fijo <code>1/60 s</code>
Animaciones de combate	6 frames cada una, con duración propia por frame (ms)
Direcciones del sprite	8: índice 0..7 = S, SW, W, NW, N, NE, E, SE (relativas a la cámara)
Equipos	<code>player</code> / <code>foe</code> . Un golpe solo afecta al otro equipo

1. Bucle y relojes

1.1 Orden de actualización de cada paso (main.js step)

- `gdt = combatStep(dt)` — avanza `ct` (0 durante el hitstop, `dt · k` en cámara lenta).
- Controles (`controlsUpdate(dt)`): cuenta de la pulsación de ataque mantenida, seguir al dedo, teclado, ataque/paseo pendientes).
- `pf.update(gdt)` — máquina de estados de combate del jugador.
- `player.update(gdt)` — movimiento del cuerpo del jugador.
- Elección de animación y dirección del sprite del jugador (`character.update`).
- Enemigos (`updateFoes(gdt)`): para cada uno, `ai.update` → `fighter.update` → `body.update` → animación → `def.update`.
- Objetivo fijado (`targetUpdate(dt)`), grupo (`groupUpdate(gdt)`): turnos, pinza, multi-parry).
- `combatAfter(dt)`: golpes pendientes por calibración, riposte (caducidad), entrenamiento.

9. Cámara y render [visual].

Nota: el hitstop congela `ct` y por tanto a los luchadores y sus cuerpos (reciben `gdt = 0`), pero no la cámara.

1.2 Hitstop y cámara lenta (combat.js)

```
combatAdvance(dt):
  if hitstop > 0: hitstop = max(0, hitstop - dt); return 0          # todo el paso congelado
  if slow.pend: slow.t, slow.k = slow.pend; slow.pend = null
  if slow.t > 0: slow.t -= dt; dt *= slow.k
  ct += dt; return dt
stop(v): hitstop = max(hitstop, v)                                # nunca acorta uno en curso
slowmo(dur, k): slow.pend = {dur, k}                             # empieza en cuanto acaba el hitstop
```

Cámara lenta tras el parry perfecto del jugador: 0.12 s reales al 30 %; tras un DOBLE/TRIPLE PERFECTO: 0.15 s al 30 %. Valores de hitstop: ver §5.6.

1.3 Instante de una pulsación (pressTime) y calibración

Toda pulsación de guardia y de ataque se sitúa en `ct` con la **marca de tiempo del evento** (`event.timeStamp`), no con el frame en que se procesa:

```
pressTime(ts, human):
  t = ct
  if ts válido:
    if paso fijo (pruebas) o sin datos del frame:
      t += clamp((ts - ahora)/1000, -0.12, +0.05) * (slow.t > 0 ? slow.k : 1)
    else:
      # tiempo real
      x = (ts - instante_real_del_último_paso)/1000
      if x < 0: t += max(-max(0.12, dur_último_frame + 0.02), x) * ritmo_del_último_paso
    else:
      D = max(x, dur_esperada_del_frame)          # media móvil (α 0.2) con tope 0.25 s, mín.
  con el último
    k = hitstop > 0 ? 0 : (slow pendiente ? su k : slow.t > 0 ? slow.k : 1)
    t += min(x, 0.25) * min(D, 0.05)/D * k      # el recorte de dt a 50 ms también se
  aplica
  return human ? t - calib/1000 : t              # la calibración, solo para el jugador
```

`ritmo_del_último_paso = min(1, gdt / tiempo_real_del_paso)` (0 durante el hitstop). **Calibración** (`calib`, ms, rango -60..+150, pasos de 5; + = tus pulsaciones llegan tarde): desplaza la ventana del parry. Además, con `calib > 0` el golpe de un enemigo se resuelve `calib` ms DESPUÉS de su impacto (para que una pulsación «tarde» pueda contar). Medida: prueba de ritmo de 10 pulsos cada 750 ms (el primero a 1 s); para cada pulso desde el 3.º se toma el toque más cercano a < 300 ms; si hay ≥ 4, `calib = mediana(toque - pulso)` redondeada a 5 ms.

En Unity: usar el tiempo del evento de entrada (Input System: `InputAction.CallbackContext.time`) y la misma conversión.

2. Movimiento (player.js; el mismo cuerpo `Player` lo usan los enemigos)

2.1 Parámetros (data/movement.json)

Parámetro	Valor	Notas
Radio del cuerpo	0.28 u	autómata 0.42, zombi 0.36, perro 0.32

Parámetro	Valor	Notas
Velocidad de marcha (jugador)	0.8-1.25 u/s según la dirección del sprite	S 1.25 · SW 1.039 · W 0.8 · NW 1.075 · N 1.25 · NE 1.128 · E 0.85 · SE 0.8 (sale de los pies dibujados, §2.6). walkSpeed = 1.05 alturas/s NO se usa con los pies anclados (por defecto)
Velocidad de carrera	$2.7 \times 1.7 = 4.59 \text{ u/s}$	autómata $\times 0.6 = 2.754 \text{ u/s}$
Aceleración	velocidad de la marcha / 0.25 s	(inercia activada por defecto; sin ella, 5 alturas/s ²)
Deceleración	$4.2 \times 1.7 = 7.14 \text{ u/s}^2$	al reducir velocidad se usa decel $\times 1.6 = 11.42 \text{ u/s}^2$
Giro máximo	11 rad/s	
Correr por distancia	camino > 4 alturas = 6.8 u → corre; vuelve a andar a < 2.2 alturas = 3.74 u del final	solo al tocar (no al seguir el dedo)
Seguir el dedo	corre si quedan > 1.2 alturas (2.04 u), anda si < 0.7 alturas (1.19 u)	
Llegada	radio 0.004 alturas al tocar, 0.18 alturas siguiendo	
Arranque desde parado	espera 0.1 s (salvo noDelay: seguir al dedo, IA)	

2.2 Seguimiento de un camino (por paso)

```

if camino:
  avanzar puntos: mientras haya ≥ 2 y dist(punto0) < (punto1.salto ? 0.12 : 0.35): quitar punto0
  si punto0 es aterrizaje de un salto y no está en una acción de combate:
    si es arista de salto o |Δaltura| > MAX_STEP: quitar y terrainJump(punto0); return
  resto = longitud restante
  marcha: (ver tabla)
  vWanted = min(vMarcha, sqrt(2·decel·max(resto - radioLlegada, 0))) # frena justo a tiempo
  si arranque pendiente: vWanted = 0
  girar el rumbo hacia el punto, como mucho 11 rad/s·dt; si el error > 1.6 rad → vWanted ×0.35;
  > 0.8 rad → ×0.7
  speed += clamp(vWanted - speed, -decel·1.6·dt, accel·dt); speed ≥ 0
  tryMove(cos·paso, sin·paso) con paso = min(speed·dt, dist)
  atasco: si en 0.45 s no se acerca 0.015 u al punto actual → re-planificar (A* sin suavizar)
  hasta 3 veces; luego rendirse
  llegada: si queda un punto, a ≤ radioLlegada y speed < 0.12·vMarcha → colocarse en el punto,
  parar
  else: speed -= decel·1.6·dt (deslizándose por inercia)

```

tryMove(dx, dz): comprueba el destino y el borde del cuerpo (radio) en la dirección del movimiento con canStep (celda libre y |Δaltura| ≤ 0.5). Si no puede, prueba solo dx y solo dz (desliza por la pared).

2.3 Altura del cuerpo y escalones

El suelo es de baldosas con alturas múltiplos de 0.5 u. El cuerpo sigue la altura del suelo con suavizado exponencial ($k = 1 - e^{-(dt \cdot 10)}$); con pies anclados 30 subiendo y 60 bajando, tomando la altura del pie apoyado). Es casi visual: lo lógico es que **se puede subir/bajar andando un desnivel ≤ 0.5 u** entre celdas vecinas.

2.4 Salto en el sitio / hacia delante (botón SALTAR, tecla L)

Solo si no hay acción de combate. prep 0.07 s → vuelo 0.62 s (altura $4u(1-u) \cdot 0.5$ alturas, máx 0.85 u) → aterrizaje 0.15 s. Si se movía (> 0.2·vMarcha) avanza a max(speed, vMarcha) y al aterrizar conserva el 65 % de la velocidad. Sirve para esquivar el BARRIDO del autómata (cuenta si la altura >

0.12 alturas = 0.204 u en el impacto).

2.5 Salto CONTEXTUAL (terreno)

- El jugador salta solo, al seguir un camino que tiene una **arista de salto** (ver §3): desnivel entre celdas vecinas **ortogonales** $> 0.5 \text{ u}$ y $\leq \text{JUMP_MAX} = 2 \cdot 0.49 \cdot 1.03 = 1.0094 \text{ u}$ (2 «cabezas» de 0.49 u, +3 %). En el mapa los desniveles van de 0.5 en 0.5, así que se saltan bordes de **1 u**; 1.5 u o más es infranqueable.
- El punto anterior al aterrizaje es el despegue: hay que pisarlo a 0.12 u.
- `terrainJump`: quieto al despegar; prep 0.11 s, vuelo $0.4 + 0.12 \cdot |\Delta h|$ s, aterrizaje 0.16 s. Posición en línea recta despegue→aterrizaje; altura: subiendo $1 - (1-u)^2$, bajando u^2 , + arco ($\Delta h > 0$? $0.25 + 0.35 \cdot \Delta h$: 0.28) $\cdot 4u(1-u)$. Sin colisiones en el aire. Velocidad 0 al aterrizar.
- Tocar una zona libre que no se alcanza ni saltando (`!path.reached`): no se mueve; marcador ROJO **[visual]**.
- Los enemigos NO saltan (A* sin aristas de salto): rodean.

2.6 Pies anclados (root motion) — qué es lógica y qué no

En el juego actual NO hay root motion por frames: el cuerpo avanza de forma continua y una deformación de la pierna apoyada mantiene la bota quieta en el suelo **[visual]**. Lo lógico es la **velocidad de la marcha**, derivada de los pies dibujados para que no patinen:

```
locoParams(walk, dir): fps = 12 (autómata 11, relleno ≈12·velocidad relativa)
  gap = mayor hueco (en frames, cíclico) entre frames de contacto
  v = art.fps/6 + 0.64·15·texel_u / f / (gap/fps)      # art, f: medidas del ciclo dibujado
  (assets/feet.json)
  v = clamp(v, 0.8, 1.25) u/s   (el relleno sin recorte)
run: velocidad = runV; frames con pesos (contacto 0.55, vuelo 1.1)
```

En Unity: con arte nuevo, usar root motion de las animaciones o fijar la velocidad para que coincida con la zancada. Conservar el rango (≈ 0.8 – 1.25 u/s andando, 4.59 u/s corriendo) para que los tiempos de llegada no cambien.

2.7 Elección de animación y dirección del sprite (character.js)

- Locomoción con histéresis de velocidad: pasa a `run` si `speed > walkV·1.35` (vuelve a `walk` si `< walkV·1.18`); a `walk` si `> 0.07 alturas/s` (vuelve a `idle` si `< 0.035 alturas/s`). Con camino y `speed > 0.01 alturas/s`, `walk`. Sin hoja `run` (relleno) → `walk`; sin `jump` → `idle`.
- Dirección** (8 sectores de 45° respecto a la cámara): en movimiento, la dirección deseada solo cambia si el rumbo se aparta más de $22.5^\circ + 8^\circ = 30.5^\circ$ del centro del sector actual (**histéresis**); parado, cambia sin histéresis. La dirección mostrada va hacia la deseada **pasando por las intermedias**, un paso cada **40 ms** (25 ms durante una acción de combate; con objetivo fijado, salta directa al atacar, guardar o esquivar). Girar la cámara 90° desplaza el índice 2.
- Al pararse tras andar/correr: 0.16 s del frame de contacto (asentamiento) **[visual]**.

3. Navegación (terrain.js, nav.js)

Cosa	Valor
Mapa	80 × 80 baldosas de 1 u, centrado en el origen (<code>x, z ∈ [-40, 40]</code>); fuera del mapa: altura 5, BORDE
Alturas	0.5 ... 7.0 en pasos de 0.5
Rejilla de navegación	2 celdas por baldosa → 160 × 160 celdas de 0.5 u

Cosa	Valor
Bloqueadas	agua y borde; obstáculos circulares (x, z, r) bloquean las celdas cuyo centro está a $< r + 0.18$
Paso andando	celda vecina libre y $ \Delta h \leq \text{MAX_STEP} = 0.5$
Vecinos	8 (coste 1 ortogonal, $\sqrt{2}$ diagonal); diagonal solo si las dos ortogonales son pasables desde el origen Y hacia el destino
Coste junto a paredes	+1.2 si hay una celda bloqueada a distancia 1 (Chebyshev), +0.4 si a distancia 2
Arista de salto (solo jugador)	vecina ORTOGONAL libre con $0.5 < \Delta h \leq 1.0094$; coste extra $\text{JUMP_COST} = 2.2$
Heurística	octil $(dx+dz) + (\sqrt{2}-2) \cdot \min(dx, dz)$; $f = g + h \cdot 1.001$
Límite	40 000 nodos (seguir al dedo 12 000; IA 6 000-14 000)

```

findPath(origen, destino, maxNodos, {jump, raw}):
  s = celda(origen); si bloqueada → la libre más cercana (anillos hasta r = 14)
  t = celda(destino); si bloqueada → la libre más cercana (penaliza  $4 \cdot |\Delta h|$  respecto a la altura del destino)
  A* con montículo binario; si no llega, devuelve el camino hasta la celda con menor h (pts.reached = false)
  puntos = centros de celda; las aristas con  $|\Delta h| > 0.5$  marcan su punto de llegada {jump: true, dh}
  último punto = destino exacto si era libre y se alcanzó
  si !raw: suavizado por línea de visión (string pulling):
    desde el punto actual, el más lejano j con lineClear(actual, j)
  lineClear(a, b): muestreo cada 0.2 u; en cada muestra y en dos líneas paralelas a  $\pm 0.12$  u, canStep entre muestras y con el borde del cuerpo (radio 0.28) por delante; y las celdas a  $\pm$ radio a los lados libres

```

`findSpot(x, z, h, clear)`: punto libre (no charco), a la altura h (± 0.3) y con `clear` u libres alrededor, buscando en anillos de 0.25 u hasta 6 u.

4. Entrada (controls.js) → órdenes

Gesto / tecla	Orden
Tocar el suelo (< 170 ms)	ir allí por A (con saltos); marcador [visual]*. En mitad de una acción de combate el paseo queda pendiente
Mantener (≥ 170 ms) o arrastrar (> 12 px, lento) sobre el suelo	SEGUIR AL DEDO: re-planifica cada 0.2 s si el punto se movió > 0.4 u (12 000 nodos); al soltar va al último punto
Deslizar rápido (> 42 px en < 260 ms)	ESQUIVA en esa dirección (pantalla → suelo deshaciendo el acortamiento isométrico)
Tocar un enemigo que NO es el objetivo	lo SELECCIONA (objetivo fijado), no ataca
Tocar el objetivo / tecla J	pulsación de ATAQUE: soltar antes de 0.4 s = LIGERO; mantener ≥ 0.4 s = FUERTE con carga (§8)
Botón GUARDIA / tecla K	<code>guardDown</code> (con la marca de tiempo) y <code>guardUp</code> : tocar = parry, mantener = bloqueo
Botón SALTAR / tecla L	salto en el sitio o hacia delante
Espacio	esquiva (dirección de WASD o hacia atrás)
WASD / flechas (+Mayús)	mover: destino 1 u por delante (3.2 u con Mayús → corre)
Tab	siguiente objetivo

- Toque sobre un enemigo: caja de pantalla del sprite (ancho = $\text{alto} \cdot 0.42 + 22$ px a cada lado; de 18 px sobre la cabeza a 16 px bajo los pies); gana el más cercano al centro.

- Ataque a un enemigo lejano (> 3.2 u y sin acción): camina hasta 1.1 u de él y ataca al llegar (≤ 2.4 u, o ≤ 3.4 u si ya no hay camino; se cancela a los 5 s).
- J sin objetivo: el enemigo más cercano a ≤ 3.4 u y ± 2.0 rad de delante; si no hay, en la dirección de WASD.
- Si al soltar el dedo era un deslizamiento rápido y había una pulsación de ataque en curso, se anula (y si cargaba, la suelta).

5. Combate: máquina de estados (fighter.js)

5.1 Estado de un luchador

hp, hpMax · st, stMax (stamina) · post, postMax (postura) · act (acción en curso o nada) · buf (búfer) · guardHeld · guardT (instante de la última pulsación de guardia, en ct) · pen (penalización del parry) · counterT, airCounterT (fin de ventanas de contraataque) · rip (ventana de riposte, §7) · flash [visual].

	Jugador	Autómata	Eco	Zombi	Perro
Vida	100	260	120	30 (3 golpes)	20 (2 golpes)
Stamina (regen/s)	100 (34)	120 (12)	100 (34)	999	999
Postura	100	100	100	100 («fatiga»)	100
Aturdido al romperla	— (ver §6.5)	3.2 s	2.4 s	2.2 s	2.2 s
Esquiva: distancia / frames invulnerables	2.3 u / 2-3	2.6 u / 1-3	2.3 / 2-3	—	—
Multiplicador de retroceso	1	0.45	1	0.9	1
Radio para recibir golpes	0.3	0.5	0.3	0.36	0.32

5.2 Acciones (act) y fin

Una acción es {name, t, tt (tiempo dentro de la hoja), f (frame 0..5), speed, ...}. El frame sale de los ms de su hoja (§16). Avanza $tt += dt \cdot speed$ (más reglas abajo). Termina al pasar del último frame ($f \geq 6$), salvo death (se queda en el frame 5), block (bucle) y stun (por tiempo).

Acción	Qué es
attack1/2/3, heavy, spin, riposte, deathblow, counter, attack (relleno)	golpes (isAtk). Frame de IMPACTO HF = primer frame «activo» de su hoja (3 en casi todos; heavy 4)
parry	pulsación de guardia
block	guardia mantenida (bucle de frames 1-2; 3 = recibe; 5 = baja, 80 ms)
dodge	esquiva
hit	golpe recibido (variantes: flinch, guardia rota gb, recoil, exposed)
stun	aturdido (postura rota; enemigos)
deflected	desequilibrado tras tu parry perfecto (enemigos)
death	muerto

Reglas de avance especiales: - **Preparación lenta** (a.slow): los frames anteriores al impacto avanzan a $speed / slow$ (sin stamina: $\times 1.35$). - **Toque recuperado** (a.prepK, §8.7): los frames anteriores al impacto avanzan a $speed \cdot prepK$. - **Recuperación larga** (a.reck): los posteriores al impacto avanzan a $speed \cdot reck$. - **Plan** de los golpes enemigos (§11.3) y **carga** del fuerte (§8.2) sustituyen el avance normal. - parry con la guardia mantenida y $tt > 0.17$ s → pasa a block ($tt = 0.07$). - Sin acción, con la

guardia mantenida y vivo → entra en `block` ($tt = 0.07$): la guardia vuelve sola tras otra acción. -

Instante exacto del impacto: al pasar al frame HF, $impactT = ct - (tt - inicio_de_HF)/velocidad$ (descuenta lo que el frame se pasó en este paso). Todas las medidas de parry usan `impactT`.

5.3 ¿Qué puede empezar ahora? (`can(tipo)`) — tabla de cancelaciones

Acción actual	attack	parry (guardia)	dodge	jump
ninguna	sí	sí	si $st \geq 9$ (½ coste)	sí
en el aire (salto)	solo si $airCounterT > ct$	no	no	no
golpe propio	combo (§8.4) o, en enemigos, $f \geq HF$ si la hoja tiene next	$f \geq 4$	frames <code>cancel.dodge</code> de su hoja (attack1/2: 4-5; attack3: 4-5; heavy: 5) y $st \geq 9$	no
<code>block</code>	sí	sí	$st \geq 9$	no
<code>parry</code>	sí si $counterT > ct$ o dentro del riposte; si no $f \geq 4$	$f \geq 3$	$f \geq 4$	no
<code>dodge</code>	$f \geq 5$	$f \geq 5$	$f \geq 5$	no
<code>hit</code>	$f \geq 5$ (nunca con guardia rota)	$f \geq 4$	$f \geq 4$	no
<code>death</code> , <code>stun</code> , <code>deflected</code>	no	no	no	no

5.4 Búfer de entrada

Una pulsación que no se puede ejecutar queda en el búfer **0.18 s** (en tiempo del luchador) y se reintenta cada paso. Una sola plaza (la última pulsación sustituye). Los ataques L/H durante un golpe del combo van a una COLA aparte (§8.4).

5.5 Hitbox y resolución del golpe (`combat.js`)

- Se resuelve UNA vez por golpe, en el primer paso en que $f \geq HF$ (con $calib > 0$ y atacante enemigo: en $impactT + calib$).
- Forma: **sector circular en el suelo** delante del atacante: alcanza a t si $d \leq reach + radioHit(t)$ y $(|ángulo - rumbo| \leq arc \text{ o } d < 0.6 + radioHit(t))$. arc es el SEMIÁNGULO en grados (attack1 $\pm 70^\circ$; spin $\pm 180^\circ = 360^\circ$).
- La ESTOCADA (mikiri) también alcanza a quien esquiva HACIA ella aunque se salga del arco ($d \leq reach + 1.5$).
- Cada objetivo del otro equipo, vivo, se resuelve por separado (un golpe puede alcanzar a varios).

```

resolve(atacante a, golpe g, objetivo t):
  si es riposte/deathblow: duelResolve (§7) ; si choque: duelClash (§7.6)
  def = t.defense(g.impactT) # §6.1
  si g.counter y def ≠ evade: def = open # el contraataque no se defiende
  si def == partial: def = block
  dmg = AT.dmg · a.dmgK · g.dmgK
  si AT.perilous: how = perilousAvoid(...) (§6.8); si how: evade/mikiri; si no: def = open
  evade → nada
  perfect/parry → §6.2 · block → §6.3 · open → golpe limpio (abajo)
golpe limpio:
  romper la racha de desvíos del atacante
  deathblow = t aturdido; counter = g.counter
  daño = t es relleno ? 10 : deathblow ? max(40, dmg·3) : counter ? dmg·1.6 : dmg
  res = t.hurt(daño, rumbo, kb = AT.kb, heavy = AT.heavy o counter o deathblow, guardBreak =
AT.throw)
  postura a t (si no es remate): daño/g.dmgK · 0.7 · (counter 1.5) · (t expuesto 1.5) · AT.postK
· g.postK
  tras un remate: t.post = 35 %
  hitstop: deathblow 0.16 · si no AT.stop · g.stopK

```

5.6 Hitstop por evento

Evento	s
attack1 / attack2 / attack3 del jugador	0.075 / 0.085 / 0.12 (fuerte 0.13; spin 0.12; nivel 2 ×1.35)
golpes del autómatas	su stop (0.09–0.15)
parry normal (ligero/pesado)	0.11 / 0.13
parry perfecto (ligero/pesado) y mikiri	0.16 / 0.18
bloqueo	0.06 · guardia rota 0.10
remate al aturdido 0.16 · deathblow del duelo 0.20 · choque 0.13 · riposte 0.07/0.09/0.13	

5.7 Recibir un golpe (hurt)

```

hp -= daño
si hp ≤ 0: death (retroceso kb·0.8)
si es enemigo y no es guardia rota:
  si está en un golpe con hyper armor (nivel ≥ 2 del autómatas) y f < HF+1: aguanta (recibe el
daño, no se interrumpe)
  si el golpe no es pesado: RESPINGO = hit que acaba en el frame 2 (IMPACT+RECOIL), retroceso
kb·0.3
si no: hit completo (mira al atacante), retroceso kb·kbK (·1.25 si pesado contra enemigo);
guardia rota: gb, speed 0.62

```

Retroceso: kb u en 0.28 s (curva $1 - (1-u)^2$), con colisiones.

6. Defensa

6.1 Medida del parry (defense(impactT))

```
si invulnerable (esquiva en sus frames invulnerables): evade
si aturdido: open
early = impactT - guardT                                # antelación de la pulsación (s)
pw = max(0.07, 0.2 - pen)                                # ventana del parry
guardando = act ∈ {parry, block}
si guardando y  $-0.004 \leq \text{early} \leq \text{pw}$ :
    early  $\leq \min(0.07, \text{pw}/2)$  ? PERFECTO : NORMAL
si act == parry y  $\text{pw} < \text{early} \leq \text{pw} + 0.12$ : PARCIAL (= bloqueo)
si act == block: BLOQUEO
si act == parry y guardia mantenida: BLOQUEO
si no: OPEN (golpe limpio)
```

- **Ventana:** 200 ms antes del impacto (hasta 4 ms después). **PERFECTO:** los últimos 70 ms (o la mitad de la ventana si el spam la ha encogido). Una pulsación de 200-320 ms antes = bloqueo; antes de eso, si se soltó, el parry (450 ms) ya acabó → golpe.
- **Spam:** una pulsación a < 0.7 s de la anterior, si la anterior NO desvió nada, suma $\text{pen} += 0.045$ (máx. 0.13); pen baja 0.12/s. (Con varios atacantes, pulsar una vez por golpe no penaliza.)
- La guardia cubre 360°.
- Una pulsación en los 300 ms posteriores a un golpe que te alcanzó → texto «TARDE x ms» (entrenamiento) **[visual]**.

6.2 Parry (normal y perfecto)

	PERFECTO	NORMAL
Daño	0	0
Postura al atacante	$\text{post} \cdot 1.33 + \text{racha}$	$\text{post} \cdot 1.0 + \text{racha}$
Postura a quien desvía	0	7 (golpe pesado 10), sin poder romperla
Stamina	0	0
Atacante	DESEQUILIBRADO 0.7 s (§7.1) — enemigo frente al jugador	retrocede (hit recoil, speed 1.35, kb 0.3)
Jugador	ventana de riposte 0.7 s (3 golpes) + cámara lenta	ventana de riposte 0.32 s (1 golpe)

Racha: desvíos seguidos del mismo atacante en la MISMA cadena (la IA marca `chainId`; sin él, a < 1.25 s) suman $+3 \cdot (n-1)$. Se pierde con un bloqueo o un golpe limpio. **Postura post por golpe** del autómatas: attack1 18, attack2 21, counter 20, breaker 26, heavy 30 (peligrosos 0). Del jugador: 38/38/48, heavy 50, spin 44, riposte 22. Si el ENEMIGO desvía al jugador: el jugador queda desequilibrado (hit recoil speed 1.0) y el autómatas lanza su COUNTER (§11.7). Contra el relleno: sin postura normal; ver §13.4.

6.3 Bloqueo y guardia rota

```
coste de stamina = (golpe quiebraguardia ? 0.65·stMax : pesado ? 36 : 22) · (atacante relleno ? 0.5 : 1)
si el golpe es «breaker» (rompeguardias del autómatas, su heavy): st = 0
si st ≤ 0: GUARDIA ROTA → hurt(daño·0.5, guardBreak) (hit gb, speed 0.62; sin acciones hasta que acabe)
si no: hp -= daño·0.15 (desgaste); frame de reacción 0.11 s; retrocede 0.12 u; postura +18 (pesado 26) — ESTA sí puede romperla
```

Stamina: se recupera a su ritmo (34/s el jugador) tras 0.55 s sin gastar; con la guardia alta al 35 %.

6.4 Postura (estilo Sekiro)

- Sube con lo de arriba. Se recupera sola: tras 1.6 s sin ganar postura (0.45 s si está bloqueando con la guardia mantenida), baja a $9/s \cdot hpK \cdot (\text{bloqueando} ? 2.8 : 1)$ con $hpK = 0.15 + 0.85 \cdot (hp/hpMax)^{1.5}$ (con 30 % de vida, ~29 % del ritmo). El relleno: retraso 6 s y 5/s.
- No baja mientras está aturdido.

6.5 Rotura de postura

- **Enemigo:** `stun` durante su `stunTime` (autómata 3.2 s); al acabar, `post = 35 %` (el relleno con aturdido corto conserva su fatiga). Golpear a un aturdido = REMATE ($\max(40, \text{daño} \cdot 3)$).
- **Jugador:** `post = 30 %`, suelta la guardia, `hit` con guardia rota (`speed 0.5` → ~0.96 s sin poder hacer nada).
- **Un parry nunca rompe la postura de quien desvía** (su coste usa `noBreak`); un bloqueo sí.

6.6 Esquiva

Coste 18 de stamina (necesita ≥ 9). Sin dirección: hacia atrás. Mira al lado contrario de hacia donde se aparta (la hoja pinta un paso atrás) o al objetivo fijado. Recorre `dodgeDist` con curva $1 - (1-u)^{2.2}$ entre la mitad del frame 1 y el final del frame 4. Invulnerable en los frames 2-3 (autómata 1-3).

6.7 Contraataques del jugador

- **Riposte** tras un parry (§7.2) — el principal.
- `counterT = ct + 0.35` tras el MIKIRI; tras un parry perfecto solo si el jugador nunca ha tenido una ventana de riposte (ver §17): el siguiente golpe que empieza en ella es indefendible, $\times 1.6$ daño, $\times 1.5$ postura.
- **Contraataque en el aire:** tras saltar un BARRIDO, `airCounterT = ct + 0.6`; atacar en el aire = `attack3` indefendible (`counter`).

6.8 Ataques PELIGROSOS (autómata): ni parry ni bloqueo

Peligroso	Respuesta	Regla
BARRIDO (sweep)	SALTAR	en el impacto, altura del salto > 0.12 alturas → evita; contraataque en el aire 0.6 s
ESTOCADA (thrust)	esquivar HACIA él (mikiri)	esquiva empezada 40-340 ms antes del impacto (o invulnerable); ángulo entre su dirección y (rumbo del atacante $+ \pi$) < 1.05 rad → MIKIRI: +42 de postura al autómata, retrocede, tu esquiva se corta y <code>counterT 0.35</code> s. Otra esquiva a tiempo: solo evita
AGARRE (grab)	esquivar de LADO	igual, con ángulo entre 0.8 y 2.35 rad; si no: te atrapa (daño 30, guardia rota)

7. Duelo: desequilibrio, riposte, deathblow, counter, choque (duel.js)

7.1 DESEQUILIBRADO (deflected)

Tras el parry perfecto del jugador (si no le rompe la postura): el enemigo entra en `deflected 0.7` s (la hoja se reescala a 0.7 s), retrocede $0.38 \cdot kbK$ u, `exposed` (tus golpes limpios le quitan $\times 1.5$ de postura), no puede actuar.

7.2 RIPOSTE

```
al parry del jugador (no relleno, sin romper postura):
  rip = {objetivo, until = ct + (perfecto ? 0.7 : 0.32), max = perfecto ? 3 : 1, n = 0}
pulsación de ataque con rip activo (ct ≤ until, n < max, objetivo vivo):
  n = n + 1
  si n == 3: err = pt - beat ; mal = (beat aún no fijado) o |err| > 0.08      # ritmo del 3.º
  si hay un riposte en curso antes de su impacto+1: queda en cola; si no: empieza «riposte»
golpe de riposte (hoja riposte: impacto a los 135 ms; sin hoja attack1 ×1.4):
  va a por el objetivo (lunge hasta 1.2 u, distancia 1.05); no se puede fintar
  si es el 3.º y mal: EL ENEMIGO LO DESVÍA (parry), el jugador +12 de postura (sin romper) y
retrocede; fin
  si no: indefendible; daño 8/10/13; postura 10/14/20 (×1.15 si expuesto); hitstop
0.07/0.09/0.13
  until = max(until, ct + 0.45); el enemigo vuelve a OFF BALANCE
  tras el 2.º: beat = impacto + 0.17 (ventana ±0.08)
  si le rompe la postura → DEATHBLOW inmediato
```

Un riposte completo desde postura 0 deja la barra en ~3/4 (no es un premio automático).

7.3 DEATHBLOW

Hoja deathblow (impacto a los 340 ms; sin hoja attack3). Indefendible, 100 de daño; si sobrevive, `post = 35 %` y `hit`.

7.4 Reacción según el peso (enemigos)

Ligero → respingo (frames 0-1 de hit); pesado → tambaleo completo con retroceso ×1.25; durante sus golpes de nivel ≥ 2 (attack2, peligrosos, breaker, heavy) → hyper armor (no se interrumpe).

7.5 COUNTER del autómatas e intercambio de desvíos (clin-clin)

- Si el autómatas desvía tu golpe: tú quedas desequilibrado; a los **0.12 s** empieza su COUNTER (carga 0.1 s + suelta 0.34 s → se puede desviar). Al acabar resopla 0.5 s.
- Si desvías su counter con un PERFECTO: con probabilidad `fail[n] = 0.3, 0.5, 0.75, 1` ($n = n.º$ de counter) FALLA él → desequilibrado 0.9 s y riposte de 3 golpes con ventana 0.9 s; si no, vuelve a contraatacar a los 0.06 s con una suelta más corta (0.34 → 0.31 → 0.28 → 0.26 s). Con un parry normal termina el intercambio.

7.6 CHOQUE

Si los dos golpes impactan a ≤ **80 ms** el uno del otro, se miran (≤ 1.6 rad) y ninguno es peligroso, counter, riposte, deathblow ni en el aire: chispas, ambos retroceden (`hit recoil speed 1.3`, `kb 0.5·kbK + 0.15`), sin daño, hitstop 0.13.

8. Ataques del jugador: ligero/fuerte, carga, combos, ritmo (moves.js, controls.js)

Estado: **todo esto está implementado y probado** (Combate completo, etapas 1-7). Pendiente: etapa 8 (interfaz móvil nueva: solo maqueta, sin implementar) y etapas 9-10 (entrenamiento de combos, bot, pruebas finales). Ver §15.

8.1 Ligero / fuerte

- Pulsar (toque sobre el objetivo, J): empieza a contar lo que se mantiene (en tiempo de paso, incluye el hitstop).

- Soltar antes de **0.4 s** → LIGERO (L).
- Al llegar a **0.4 s** mantenido → empieza el FUERTE (H) cargando ($pressCt = ct - \text{mantenido}$). Si el objetivo está a $> 3.4 u$ y no hay acción, no carga (al soltar irá a por él).

8.2 Carga del fuerte (hoja **heavy**: **CROUCH, CHARGE, CHARGE MAX, RELEASE, IMPACT, RECOVERY**)

- Mientras carga: frames de carga [1, 2] en bucle; la stamina no se recupera; ERES VULNERABLE (cualquier golpe la interrumpe).
- A **1.2 s** desde la pulsación: NIVEL 2 ($\times 1.5$ daño, $\times 1.6$ postura, $\times 1.35$ hitstop, coste $\times 1.55$).
- Soltar (o **1.8 s**: se suelta sola) → salta a RELEASE: impacto 60 ms después.
- Guardia durante la carga = FINTA (cancela, -14 stamina, la pulsación sigue como guardia).

8.3 Stamina de los golpes

Paso	Coste	Paso	Coste
L	6	H	18
LL	7	LH (quiebraguardia)	20
LLL	12	LLH (remate giratorio)	22
HL (corte de salida)	8	HH (cargado doble)	26

Nivel 2: $\times 1.55$. Sin stamina suficiente el golpe sale igual pero con la preparación $\times 1.35$ más lenta.

8.4 Combos en datos (tabla **STEPS**)

Secuencia	Hoja	Efecto	¿Cierra?	Pulso del siguiente
L	attack1	—	no	0.16 s tras el impacto
LL	attack2	—	no	0.17 s
LLL «Combo básico»	attack3	—	sí	—
H	heavy	carga	no	0.24 s
LH «Quiebraguardia»	heavy	carga; si te lo bloquean: -65% de su stamina; postura $\times 1.25$	sí	—
LLH «Remate giratorio»	spin	360° (alcanza a varios), recuperación $\times 0.55$ de velocidad	sí	—
HL «Corte de salida»	attack2	velocidad $\times 1.35$, daño $\times 0.9$	sí	—
HH «Golpe cargado doble»	heavy	carga; velocidad $\times 0.8$, daño $\times 1.2$, postura $\times 2.2$, recuperación $\times 0.75$	sí	—

```

comboSeq(kind, pt):
  continúa si: (golpe en curso del combo, sin cerrar, y ( $f \geq HF$  o es la 2.ª de la cola))
               o (sin acción y  $pt \leq fin\_del\_último\_golpe + 0.35$ )
  seq = combo.seq + kind si existe en STEPS; si no, kind (empieza de nuevo)
  cola: una pulsación L/H durante un golpe del combo (que no cierre y no esté cargando) va a la
        cola (máx. 2);
  el siguiente golpe empieza al entrar en el frame HF+1 (FOLLOW THROUGH) del actual
  ritmo: si continúa un combo y existe beat:  $err = pt - beat$ ; EN RITMO si  $|err| \leq 0.05 \rightarrow$  daño y
        postura  $\times 1.2$ 

```

8.5 Atracción (lunge) y giro

Al empezar un golpe: objetivo = el fijado (si está a ≤ 6 u; gira al instante) o el enemigo más cercano a ≤ 3.4 u y ± 2.0 rad. Avanza `lunge = clamp(d - want, 0, cap)` (want 1.0, attack3 1.15; cap 0.9, attack3 1.6) durante los frames `HF-2..HF+1` (desde RELEASE en el fuerte), sin acercarse a < 0.85 u. Sin objetivo: 0.25 / 0.35 / 0.9 u al aire. Gira hacia el objetivo a `dt · 22` de la diferencia.

8.6 Daño de los golpes del jugador

Golpe	Daño	Alcance	Arco $\pm$	Retroceso	Postura (si se lo desvían)	Pesado
attack1	10	1.6	70°	0.35	38	no
attack2	12	1.65	70°	0.35	38	no
attack3	22	1.95	85°	0.6	48	sí
heavy	22 (postK 1.5)	2.05	80°	0.75	50	sí
spin	15 (postK 1.7)	2.25	180°	0.6	44	sí
riposte	8/10/13 (§7.2)	1.95	80°	0.2	22	no
deathblow	100	2.2	120°	0.6	—	sí

8.7 Toque recuperado (arranque del ligero)

El ligero sale al SOLTAR; si empieza en el mismo paso en que se suelta, su preparación se acelera para recuperar lo que se mantuvo: `cut = min(mantenido, 0.4 · preparación)`, `prepK = prep / (prep - cut)`. Pulsar → impacto con un toque de 100 ms: 233 ms (sin esto, 317).

8.8 Lo que NO está o está a medias

- **Interfaz móvil nueva** (etapa 8): solo hay una maqueta; los controles son los del §4.
- **Entrenamiento de combos, bot de combos** (etapas 9-10): no hechos.
- El «RETRASO» (mantener el ataque para retener la preparación) sigue en la máquina de estados (`data.hold`) pero ningún control lo envía ya: lo sustituyó la carga del fuerte.

9. Fijar objetivo (target.js)

- Tocar un enemigo lo selecciona; tocar el objetivo ataca; Tab: siguiente vivo a ≤ 9 u, del más cercano al más lejano.
- Con objetivo: al atacar (si está a ≤ 6 u), guardar, hacer parry o esquivar el jugador se gira AL INSTANTE (rumbo y sprite); mientras guarda/bloquea sigue mirándolo; al caminar mira hacia donde va.
- Se suelta: a > 9 u; sin línea de visión > 1.5 s; si muere (pasa al enemigo vivo más cercano a ≤ 9 u que esté a ≤ 6 u o persiguiéndote).
- Línea de visión: a la altura del pecho (1.1 u), la cortan el terreno más alto y los obstáculos altos (árboles, ruinas...; no troncos caídos, lápidas, rocas ni cuerpos).
- Cámara: se desplaza hacia el objetivo un 22 % de la distancia (máx. 2 u) **[visual]**.

10. Combate en grupo (group.js)

- **Turnos**: como mucho **2** enemigos atacan a la vez. Un enemigo pide turno al empezar su cadena y lo suelta al acabarla (o a los 2.5 s sin empezar). Los demás RODEAN al jugador a 3.6 u (cada uno cerca de su ángulo, separados). «En grupo» = 2 o más enemigos persiguiendo a < 9 u.

- **PINZA:** cada 0.6 s, si hay ≥ 2 autómatas libres a < 3.4 u a lados opuestos ($> 100^\circ$) y nadie tiene turno, con probabilidad 0.35 (si no, espera 1.2 s): los dos golpean con un zarpazo cuyo impacto cae a $ct + 1.05$ (el segundo ± 40 ms); enfriamiento 4 s.
- **MULTI-PARRY:** los parries de la MISMA pulsación cuyos impactos caen en 0.15 s forman grupo (la guardia sirve para todos). Todos perfectos = DOBLE/TRIPLE PERFECTO: cámara lenta 0.15 s al 30 %, +15 de postura a cada uno y todos DESEQUILIBRADOS; el riposte va al objetivo fijado si es uno de ellos. Impactos más separados: un parry por golpe sin penalización por spam.
- Flechas de borde (golpe por la espalda $> 110^\circ$ o fuera de pantalla) **[visual]**.

11. IA del Autómata del bosque (enemies/automaton/automaton.js)

Datos: `data/automaton.json`. Dificultad: `data/difficulty.json` (react 1, parry 0.3, block 0.35).

11.1 Estados

```
patrol: pasea a 1..4.2 u de su casa (pausas 1.8-4 s); → chase si ve al jugador (< 8 u) o ha perdido vida
chase: se gira hacia ti (< 3.6 u, dt·5); se acerca por A* a 1.7 u de ti (corre si > 4.5 u; replanifica cada 0.4 s)
      ataca si: cool ≤ 0, no resoplando, no esperando un golpe leído, no pasivo, canStrike() y d ≤ 2.6 y tiene turno
home: si el jugador se aleja > leash+3 = 18 u de su casa (o muere): vuelve andando
```

`canStrike()`: el jugador vivo y no en `hit` ni `death` (nunca ataca si estás en el suelo o encajando un golpe).

11.2 Cadenas

`pickChain` por peso (o la forzada en entrenamiento). Cada paso: [golpe, ritmo, pausa].

id	nombre	peso	pasos
rrl	rápido-rápido-lento	3	attack1 f · attack1 f +0.05 · attack2 s +0.2
lpr	lento-pausa-rápido	3	attack2 s · attack1 f +0.6
dos	zarpazo y barrido	3	attack1 n · attack2 n +0.1
cuatro	cuatro golpes	2	attack1 f · attack1 n +0.08 · attack2 f +0.25 · attack1 s +0.05
barrido	zarpazo y barrido bajo	1.2	attack1 n · sweep n +0.2
estocada	barrido y estocada	1.2	attack2 n · thrust n +0.3
agarre	zarpazo y agarre	1.2	attack1 f · grab n +0.25
pesado	zarpazo y golpe pesado	2	attack1 n · heavy s +0.15
pesado2	golpe pesado y zarpazo rápido	1.2	heavy s · attack1 f +0.3

```

runChain (cada paso):
  si muerto, aturdido o el jugador muerto: endChain(false)
  si recibe un golpe sin recoil: endChain(false); si es recoil (desviado): sigue
  si deflected: endChain(false)
  si el paso actual es una finta y la carga ha terminado (feintNow): cortar y strike(golpe
alternativo, ritmo "feint")
  esperar al frame HF+1 del golpe actual; en el ÚLTIMO golpe, recuperación a ×0.7 de velocidad y
fin al acabar
  esperar pausa·pauseK desde ahí; si no puede golpear (canStrike) 1.2 s → endChain(false)
  siguiente golpe
endChain(limpia): suelta el turno; ficha «E» para su modelo; si limpia: RESOPLA 0.95 s (counter:
0.5 s) – ni ataca ni se
  defiende –; luego pausa (0.9..1.6 s)·pauseK (entrenamiento 1.1 s)

```

11.3 Plan de un golpe (aviso legible)

```

strike(golpe, ritmo, paso):
  plan = {wind = W[ritmo], hold = paso.retraso (truco) o 0, rel = moves[golpe].rel, feint =
paso.finta}
  si el golpe es heavy: hold = max(hold, U(0.35, 0.55))
  W = {f: 0.12, n: 0.32, s: 0.55, feint: 0.16, c: 0.10}
  el avance del plan sustituye al de la hoja:
    CARGA      t < wind:          tt = t/wind · inicio(HF-1)          (frames 0..HF-2; el ojo
parpadea)
    RETENCIÓN  wind ≤ t < wind+hold: tt congelado al final de la carga (ojo fijo)          → evento
"hold"
    SUELTA     hasta + rel:          frame HF-1 en «rel» segundos (destello + chasquido) → evento
"release"
    IMPACTO    en wind+hold+rel; luego la recuperación de la hoja a velocidad normal
finta: al acabar la carga (t ≥ wind) se congela, la cadena la corta y lanza el golpe alternativo
(attack1→attack2) con
  ritmo "feint" (carga 0.16 s): su aviso vuelve a empezar

```

golpe	hoja	nivel de aviso	suelta (s)	preparación total f / n / s
attack1	attack1	1	0.36	0.48 / 0.68 / 0.91
attack2	attack2	2	0.38	0.50 / 0.70 / 0.93
sweep (barrido, salto)	attack2 (agachado)	3	0.46	0.58 / 0.78 / 1.01
thrust (estocada, mikiri)	attack1 (pose de dodge)	3	0.46	0.58 / 0.78 / 1.01
grab (agarre, de lado)	attack1 (pose de block/parry)	3	0.46	0.58 / 0.78 / 1.01
counter	counter	1	0.34 (intercambio 0.31/0.28/0.26)	carga 0.1 → 0.44
breaker (rompeguardias)	attack2	2	0.42	0.54 / 0.74 / 0.97
heavy	heavy	2	0.40	0.87-1.07 / 1.07-1.27 / 1.30-1.50 (retención 0.35-0.55)

Golpes retrasados: + retención 0.32-0.50 s. Nivel de aviso **[visual]**: 1 cian, 2 más fuerte, 3 ROJO + sonido grave. Distancia preferida (*want*) y estocada máxima (*cap*): attack1 1.55, attack2 1.75, sweep 1.6/1.3 (persigue), thrust 0.95/2.3 (desde la suelta, busca a 4.4 u), grab 1.2/2.4 (persigue), counter 1.5, breaker/heavy 1.75/1.8. Daños en `data/automaton.json`.

11.4 Trucos (como mucho uno por cadena y nunca en dos cadenas seguidas)

Probabilidad $0.4 \cdot (1 + 0.5 \cdot \max(0, \text{adapt}))$; mitad RETRASO (retención 0.32-0.5 s), mitad FINTA (attack1↔attack2), sobre un paso no peligroso al azar.

11.5 Ramas según tu respuesta al 1.er golpe de la cadena

Tu respuesta	El 2.º golpe pasa a ser
lo desvías	RETRASADO (cuenta como el truco; si ya llevaba uno, sigue igual)
lo esquivas (o falla por tu esquiva)	ESTOCADA (ritmo n, +0.15 s)
lo bloqueas (o guardia rota)	HEAVY (ritmo s, +0.25 s)
te alcanza / choque	sigue igual

(No se ramifica en entrenamiento ni en counter/intercambio.)

11.6 Defensa: te LEE (modelo de trigramas), no tira dados

```
fichas: cada impacto de tu golpe (a1/a2/a3/aH/aS), cada finta tuya (F), el fin de su cadena (E)
y una pausa larga (., si
  pasan > 2.5 s). Cada ficha de ataque lleva el ritmo desde la anterior: q < 0.45 s, m < 1 s, s
más (clave = tipo+ritmo).
addToken(k, t):
  para ctx en [trigrama (2 anteriores), bigrama (la anterior)]:
    g = grams[ctx]; todos los g.next[*].n *= 0.8 # memoria reciente
    g.next[k].n += 1; seen++; gap medio = media de las últimas 3
  máx. 40 fichas; si es la 1.ª tras ≥ 0.8 s sin golpear → apertura (abajo); forecast(t)
forecast(t):
  g = trigrama si n ≥ 1.7, si no bigrama si n ≥ 1.7; si no, nada
  mejor = next con más peso; conf = n_mejor / n_total; prevé el instante t + gap
  (en entrenamiento o con combos L/H, manda el modelo de combos: no se compromete aquí)
  si mejor es un ataque, seen ≥ 2 y conf ≥ min(1 - 0.5·dif.parry, 0.9 - 0.6·dif.block):
    apertura (tras E o ., ritmo s o gap > 0.8) → BLOQUEO; si no y conf ≥ 1 - 0.5·dif.parry →
PARRY; si no BLOQUEO
  compromiso = {tipo, at, guardia visible a at - U(0.28, 0.34), pulsación del parry a at -
0.09}
defendStep (cada paso, si libre para defender: sin acción/en guardia/hit leve desde su frame
1/fin de su golpe sin cadena,
  sin resoplar, no expuesto, no aturdido):
  caduca a at + 0.3
  si ya toca alzar la guardia y tú estás en hit/death: cancela
  al alzarla: si es parry y quedan < 150 ms hasta su pulsación → pasa a BLOQUEO (justicia);
guardia (ojo ámbra) hasta at + 0.25
  parry: pulsa en at - 0.09 (guardDown/guardUp → acción parry)
  su parry no encuentra golpe 0.22 s después de pulsar → EXPUESTO 0.6 s (hit sin retroceso,
speed 0.55; tus golpes ×1.5 postura)
aperturas de tu combo: mide cuánto tardas en abrir tu combo desde que quedas libre (acabas tu
acción); si las 3 últimas varían
  < 0.15 s, al quedar libre de nuevo prevé la apertura y la BLOQUEA (alza la guardia entre 0.36
y 0.17 s antes)
```

11.7 Reacción humana

Al empezar tu golpe ($a < 3.6$ u): reacciona a los $rt = \text{clamp}(0.25 - 0.04 \cdot \text{clamp}((\text{react} - 0.5)/1.1, 0, 1) \pm 0.02, 0.2, 0.26)$ s. - Si es una CARGA de fuerte: te tiene muy leído y tu combo acaba en fuerte (familiaridad alta) y estás a < 3 u → te INTERRUMPE con un zarpazo rápido (carga 0.12 s); si no, alza la guardia y aguanta (hasta 2.2 s). - Si no, si tu golpe aún tarda en llegar: con < 35 % de vida o tras encajar 2 golpes en 2.2 s y si quedan > 0.12 s y stamina > 10 → ESQUIVA de lado ($\pm 90^\circ + 27^\circ$); si

quedan > 0.05 s y `block > 0` → BLOQUEA. Nunca hace parry por reacción (solo lo leído). -
Consecuencia: el zarpazo del jugador (205 ms de preparación) es más rápido que su reacción: solo lo para si lo ha leído.

11.8 Familiaridad con tus COMBOS (L/H)

```
firma = secuencia + ritmo: "LLL:r" (todo en ritmo), ":x" (todo fuera), ":m" (mezcla), ":-" (un golpe)
al cerrar tu combo (o 1.4 s sin seguirlo): fam[firma] = f + 0.25·(1 - f); las demás ×0.75;
olvido continuo  $e^{(-dt/40)}$ 
  en grupo: los demás autómatas suben fam[firma] el 50 % de lo que subió
nivel:  $\geq 0.55$  alta ·  $\geq 0.38$  media ·  $\geq 0.2$  baja
al empezar tu golpe del combo: el combo más familiar que empieza como el tuyo (cpred); si tiene nivel, ojo ámbar [visual]
al impactar tu golpe: prevé el siguiente en pulso + tu desfase habitual en ese paso + preparación de esa hoja
  alta: PARRY al golpe final, BLOQUEO a los demás · media: BLOQUEO · baja: BLOQUEO con prob. fam·1.6
  (si el siguiente es un fuerte no lo prevé: carga imprevisible; ver interrupción en 11.7)
```

Variar el combo, el ritmo o fintar lo engaña. Su conocimiento se reinicia al reaparecer.

11.9 Dificultad adaptativa

`adapt` $\in [-1.2, 1]$: -0.35 por cada muerte del jugador; $+0.12$ por cada parry perfecto a partir del 2.^o seguido; vuelve a 0 a $0.004/s$. Pausas $\times (1 - 0.25 \cdot \text{adapt})$; trucos $\times (1 + 0.5 \cdot \max(0, \text{adapt}))$.

11.10 Muerte y reaparición

Muerto: en el suelo 3 s, se desvanece 2.5 s (esporas **[visual]**) y queda oculto hasta REAPARECER (botón / respawnAll): vuelve a su casa con todo lleno, modelo reiniciado (las muertes seguidas del jugador se conservan). Aparece en 3 sitios: junto al titán (`findSpot(-8, 1)`), bosque de raíces y ruinas.

12. IA del Eco (enemigo de prueba)

Dormido en su claro; despierta a < 4.5 u o si pierde vida. Persigue por A hasta 1.45 u (corre si > 5 u; cada 0.35 s). Ataca a ≤ 2.4 u (si el jugador no está muerto y tiene turno): a > 1.75 u, 70 % `attack3` (salto); cerca, 25 %; si no `attack1` y 35 % de encadenar `attack2` (al frame 3). Espera $0.9 + U(0, 0.7)$ (+0.4 tras `attack3`) y se aparta 0.8 u durante 0.5 s. Vuelve a casa si se aleja > 14 u. Preparación $\times 1.7$ (`prepK`), daño $\times 1.3$. Aviso al empezar el WIND UP [visual]*. Usa la tabla de golpes del jugador (`attack1-3`) y las mismas reglas de combate.

13. Relleno: ZOMBI y PERRO ZOMBI (enemies/fodder/fodder.js)

Pensados para practicar el parry: un solo golpe, SIEMPRE con el mismo ritmo, sin lectura, sin trucos, sin cadenas. La facilidad sale del enemigo y de las ayudas, **nunca de agrandar la ventana del jugador**.

13.1 Datos

	Zombi	Perro
Altura	$1.05 \times \text{CHAR_H}$	$0.6 \times \text{CHAR_H}$
Vida	3 golpes (30)	2 golpes (20)
Velocidad	$0.6 \times \text{tu marcha}$	$1.3 \times \text{tu marcha}$

	Zombi	Perro
WINDUP (inicio → IMPACT, exacto)	0.8 s (carga 0.62 + suelta 0.18)	0.6 s (0.4 + 0.2; salta en la suelta)
Recuperación (IMPACT → fin)	1.0 s	0.9 s
Alcance / arco ± / daño	1.7 / 80° / 6	1.45 / 75° / 6
Distancia de ataque (engage)	1.65	2.6
Ve / correa	7.5 / 12	9 / 14
Retroceso del aturdido	0.3 u	1.5 u (en 0.28 s)
Espera entre golpes	0.6 s	0.5 s

13.2 Estados

idle → wander → chase (A*) → windup → attack (golpe en arco SOLO en el frame IMPACT) → recovery → (stun | hit | death). - chase si a < see, ha perdido vida o está en alerta (salvo los 6 s tras rendirse). Pasea a 0.8-3.2 u de su casa. - Persecución: A* hasta tu posición (si no hay, hasta un punto libre a 0.75·engage); si el final del camino queda a > engage+0.6 de ti durante > 4 s (no llega: estás en un saliente) se RINDE y vuelve. No salta. - Golpe: a ≤ engage, con la espera cumplida y con TOKEN (§13.5). - Solo existen idle, walk, attack, hit, death: nunca pide run, parry, block ni dodge.

13.3 Muerte y reaparición

En el suelo 1.6 s, se desvanece 1.6 s, oculto; reaparece en su sitio a los 9 s si no estás a < 6 u. En el grupo de práctica: vuelve a ~5 u del jugador 1 s después de desvanecerse, ya en alerta.

13.4 Recompensas y FATIGA

Tu defensa	Resultado
Parry PERFECTO	REMATADO al instante (muere; la barra de fatiga se llena)
Parry NORMAL	fatiga +50: si se llena → ¡AGOTADO! 2.2 s; si no → aturdido 1.2 s (conserva la fatiga). Retroceso (perro 1.5 u)
Bloqueo	la MITAD de stamina que contra otros; no aturde
Golpe recibido	6 de daño
Tu golpe	10 por golpe (zombi 3, perro 2); aturdido → cualquier golpe lo mata

Fatiga: empieza a bajar 6 s después del último parry, a 5/s. Su ritmo de ataque NO cambia con la fatiga.

13.5 Ayudas: anillo de timing y andamio

- **Anillo** (lógica de la ayuda): $\text{radio } r_0 \cdot \text{clamp}(\text{toImpact/windup}, 0, 1)$ (r_0 zombi 1.05, perro 0.85) → toca el centro en el IMPACT. Nivel 2: visible todo el windup; nivel 1: solo los últimos 250 ms; nivel 0: sin anillo (pose + gruñido + clic en el IMPACT, que suena en todos los niveles).
- **Andamio por tipo** (modo auto): 4 parries seguidos bajan un nivel; 3 fallos seguidos lo suben. Modo «siempre» = nivel 2; «nunca» = 0.

13.6 Grupos: TOKEN ÚNICO (metrónomo)

solo uno en WINDUP/ATTACK a la vez:
 el turno se suelta al acabar su RECOVERY o su STUN (o al morir); el siguiente empieza su WINDUP 0.7 s después
 siguiente = quien lleva más tiempo sin atacar (y a igualdad el más cercano), a ≤ 9 u
 el siguiente se coloca a su distancia y espera; los demás RODEAN a 2.5 u (avanzan ~1.05 rad/s alrededor de ti, separándose si están a < 1 rad)

Grupos del mundo: cementerio (2 zombis + perro), charcas (2 perros + 2 zombis; zombi + perro). Aparecen a < 24 u y se retiran a > 42 u si no pelean. Grupo de práctica: 4 (zombi, perro, zombi, perro) junto al jugador.

14. Entrenamiento (practice.js)

- Elegir una cadena del autómata (o «+delay», «+feint», «riposte», «fodder»): el autómata más cercano la repite sin leerse ni defenderse; nadie muere (si la vida baja del 35 %, se rellena).
- Indicador tras cada golpe: «PERFECTO/PARRY · x ms antes», «BLOQUEO», «PRONTO · x ms», «TARDE · x ms», «SIN GUARDIA»; en los peligrosos la respuesta correcta; contra el relleno, también el nivel del anillo; en «riposte», el ritmo del 3.º.

15. Reglas de justicia (y las pruebas que las verifican)

Regla	Valor	Prueba
Preparación visible de todo golpe enemigo	≥ 350 ms (mínimos reales: zarpazo rápido 0.48 s; counter 0.44 s; 4.º counter del intercambio 0.36 s). La suelta avisa ≥ 360 ms en los golpes de cadena (counter 0.34 s; intercambio 0.31/0.28/0.26 s)	parry2 «justicia...», duel3 «justicia...»
Trucos	como mucho 1 por cadena, nunca en 2 cadenas seguidas; la rama no añade otro	parry2, duel3
Nunca ataca si estás en el suelo o encajando un golpe		parry2
Ventana de castigo tras cada cadena	~1 s resoplando (0.95 s; counter 0.5 s)	parry2
Reacción del enemigo	200-260 ms, nunca < 200 ms; solo bloquea por reacción	parry2
Su parry se ve venir	guardia ≥ 150 ms antes de su pulsación	parry2
Bot humano (reacción 250 ± 40 ms) contra el autómata	desvía ≥ 75 % de cada golpe normal; evita ≥ 75 % de cada peligroso con la respuesta correcta	parry2
Riposte	un bot con buen ritmo encadena los 3 en ≥ 60 % de sus perfectos; al azar ≤ 40 %	duel3
Relleno	≥ 99 % de golpes desviados por una reacción humana (reacción + 50 ms $\leq$ windup); con anillo, ≥ 99 % desviados	fodder_bot
Turnos de grupo	nunca más de 2 autómatas atacando; nunca 2 rellenos en WINDUP/ATTACK	cc_e7, fodder_e4
La ventana del jugador nunca se agranda	200 ms / perfecto 70 ms	parry2 «niveles»
Cronometraje en tiempo real	error de la pulsación $\leq \sim 16$ ms a 14 fps	timing_rt

Resultados de referencia del bot del relleno: nivel 2 → 100 % desviados (68-69 % perfectos); nivel 0 → 97-98 % (54-57 %); autómata → 96.7 % (9.3 % perfectos).

16. Datos de animación

Lógica (hay que respetarlo en Unity aunque cambie el dibujo): **duración de cada frame (ms)**, **frame de IMPACTO**, **frames de carga**, **frames invulnerables**, **ventanas de cancelación / encadenado**, **bucle de la guardia** y **frame de reacción**. Visual: etiquetas de pose, tamaño del frame, pivote,

posición del ojo/pies, poses compuestas (show) de los peligrosos. Los JSON completos están en `data/animations/`. Columnas: `inicio` = ms desde el comienzo de la animación.

16.1 Jugador — locomoción (assets/atlas.json)

Bucles por fps (idle 6, walk 10, run 12, jump 10). Con los pies anclados la cadencia real de walk/run es 12 fps (§2.6). El salto usa sus propios tiempos (§2.4/§2.5): frame 0 preparación; vuelo $u < 0.2 \rightarrow 1$, $< 0.45 \rightarrow 2$, $< 0.72 \rightarrow 3$, si no 4; 5 aterrizaje.

Frame 120×136 px, pivote [60, 125] **[visual]**. Direcciones: S, SW, W, NW, N, NE, E, SE.

idle — 6 fps, 6 frames, en bucle.

walk — 10 fps, 6 frames, en bucle.

run — 12 fps, 6 frames, en bucle.

jump — 10 fps, 6 frames, una vez (con los tiempos propios del salto, ver arriba).

16.2 Jugador — combate (assets/combat_atlas.json)

El frame de IMPACTO de cada golpe es el primero «activo» (heavy: 4; spin: 2). La guardia (block) hace bucle 1-2 mientras se mantiene (frame 0 los primeros 60 ms; luego alterna cada 90 ms), frame 3 al recibir (110 ms) y 5 al bajarla (80 ms). parry → block si se mantiene la guardia y $tt > 170$ ms. hit: con «flinch» acaba al llegar al frame 2.

Frame 128×144 px, pivote [64, 132] **[visual]**. Direcciones: S, SW, W, NW, N, NE, E, SE.

attack1 — total 485 ms; IMPACTO en el frame 3 (a los 205 ms); «next» de la hoja (encadenado de los enemigos y del riposte): attack2.

frame	pose [visual]	ms	inicio	fase	lógica
0	ANTICIPATION	80	0	prep	
1	WIND UP	70	80	prep	
2	SWING	55	150	prep	
3	IMPACT	70	205	activo	ACTIVO, IMPACTO , encadenar
4	FOLLOW THROUGH	90	275	rec	→esquiva, encadenar
5	RECOVERY	120	365	rec	→esquiva, encadenar

attack2 — total 485 ms; IMPACTO en el frame 3 (a los 205 ms); «next» de la hoja (encadenado de los enemigos y del riposte): attack3.

frame	pose [visual]	ms	inicio	fase	lógica
0	ANTICIPATION	80	0	prep	
1	WIND UP	70	80	prep	
2	SWING	55	150	prep	
3	IMPACT	70	205	activo	ACTIVO, IMPACTO , encadenar
4	FOLLOW THROUGH	90	275	rec	→esquiva, encadenar
5	RECOVERY	120	365	rec	→esquiva, encadenar

attack3 — total 770 ms; IMPACTO en el frame 3 (a los 360 ms).

frame	pose [visual]	ms	inicio	fase	lógica
0	ANTICIPATION	130	0	prep	

frame	pose [visual]	ms	inicio	fase	lógica
1	LEAP	120	130	prep	
2	OVERHEAD	110	250	prep	
3	IMPACT	110	360	activo	ACTIVO, IMPACTO
4	FOLLOW THROUGH	130	470	rec	→esquiva
5	RECOVERY	170	600	rec	→esquiva

parry — total 450 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	GUARD UP	45	0	prep	
1	DEFLECT	55	45	activo	ACTIVO
2	SPARK	70	100	activo	ACTIVO
3	PUSH BACK	80	170	rec	
4	READY	90	250	rec	→ataque, →esquiva
5	RECOVERY	110	340	rec	→ataque, →esquiva

block — total 520 ms; bucle de los frames [1, 2].

frame	pose [visual]	ms	inicio	fase	lógica
0	RAISE	60	0	prep	
1	HOLD	90	60	activo	ACTIVO, soltar, →esquiva
2	HOLD	90	150	activo	ACTIVO, soltar, →esquiva
3	IMPACT	110	240	activo	ACTIVO, recibe, soltar
4	HOLD	90	350	activo	ACTIVO, soltar, →esquiva
5	LOWER	80	440	rec	

dodge — total 400 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	CROUCH	45	0	prep	
1	PUSH OFF	55	45	prep	
2	DASH	70	100	activo	ACTIVO, invulnerable
3	DASH	70	170	activo	ACTIVO, invulnerable
4	LANDING	70	240	rec	
5	RECOVERY	90	310	rec	→ataque, →esquiva

hit — total 480 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	IMPACT	60	0	activo	
1	RECOIL	70	60	rec	
2	STAGGER	80	130	rec	
3	STAGGER	80	210	rec	
4	RECOVER	90	290	rec	→esquiva
5	READY	100	380	rec	→esquiva, →ataque

death — total 1030 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	HIT	90	0	activo	
1	STAGGER	110	90	rec	
2	KNEES	130	200	rec	
3	COLLAPSE	140	330	rec	
4	DOWN	160	470	rec	
5	DOWN	400	630	rec	

riposte — total 385 ms; IMPACTO en el frame 3 (a los 135 ms); «next» de la hoja (encadenado de los enemigos y del riposte): riposte; sustituto sin hoja: attack1 ×1.4.

frame	pose [visual]	ms	inicio	fase	lógica
0	PARRY POSE	40	0	prep	
1	SNAP	45	40	prep	
2	LUNGE	50	85	prep	
3	CUT	60	135	activo	ACTIVO, IMPACTO , encadenar
4	CUT FOLLOW	80	195	rec	→esquiva, encadenar
5	READY	110	275	rec	→esquiva, encadenar

deathblow — total 850 ms; IMPACTO en el frame 3 (a los 340 ms); sustituto sin hoja: attack3.

frame	pose [visual]	ms	inicio	fase	lógica
0	STEP IN	110	0	prep	
1	RAISE	140	110	prep	
2	PLUNGE	90	250	prep	
3	IMPACT	170	340	activo	ACTIVO, IMPACTO
4	PULL OUT	150	510	rec	
5	RECOVER	190	660	rec	

heavy — total 770 ms; IMPACTO en el frame 4 (a los 440 ms); la carga hace bucle en 1-2 mientras se mantiene; al soltar salta al frame 3 (RELEASE): impacto 60 ms después.

frame	pose [visual]	ms	inicio	fase	lógica
0	CROUCH	120	0	prep	
1	CHARGE	130	120	carga	carga (bucle)
2	CHARGE MAX	130	250	carga	carga (bucle)
3	RELEASE	60	380	prep	
4	IMPACT	120	440	activo	ACTIVO, IMPACTO , encadenar
5	RECOVERY	210	560	rec	→esquiva, encadenar

spin — total 620 ms; IMPACTO en el frame 2 (a los 150 ms); remate giratorio: arco ±180°.

frame	pose [visual]	ms	inicio	fase	lógica
0	WIND UP	80	0	prep	
1	TWIST	70	80	prep	
2	SPIN	70	150	activo	ACTIVO, IMPACTO , 360°
3	SPIN	70	220	activo	ACTIVO, 360°
4	SLASH END	90	290	rec	

frame	pose [visual]	ms	inicio	fase	lógica
5	RECOVERY	240	380	rec	

16.3 Autómata (assets/enemy_atlas.json) — 1.4 × CHAR_H

Sus golpes NO siguen estos ms en la preparación: el PLAN (§11.3) estira los frames 0..HF-2 a la carga, congela en la retención y dibuja el frame HF-1 durante la suelta; desde el IMPACTO, la hoja a velocidad normal (último golpe de la cadena ×0.7). Esquiva invulnerable en los frames 1-3. Aturdido: frames 2-3 de hit alternando a 2.2 Hz. Poses compuestas de los peligrosos **[visual]**: estocada = dodge 0-4 + attack1 5; agarre = block 0-1, parry 1, attack1 3-5; barrido = attack2 agachado.

Frame 180×145 px, pivote [90, 130] **[visual]**. Direcciones: S, SW, W, NW, N, NE, E, SE.

idle — total 1020 ms; bucle.

frame	pose [visual]	ms	inicio	fase	lógica
0	REST	170	0	rec	
1	RISE	170	170	rec	
2	PEAK	170	340	rec	
3	FALL	170	510	rec	
4	SETTLE	170	680	rec	
5	REST	170	850	rec	

walk — total 750 ms; bucle.

frame	pose [visual]	ms	inicio	fase	lógica
0	RIGHT CONTACT	125	0	rec	
1	DOWN	125	125	rec	
2	PASSING	125	250	rec	
3	LEFT CONTACT	125	375	rec	
4	DOWN	125	500	rec	
5	PASSING	125	625	rec	

run — total 570 ms; bucle.

frame	pose [visual]	ms	inicio	fase	lógica
0	RIGHT CONTACT	95	0	rec	
1	PUSH OFF	95	95	rec	
2	FLIGHT	95	190	rec	
3	LEFT CONTACT	95	285	rec	
4	PUSH OFF	95	380	rec	
5	FLIGHT	95	475	rec	

attack1 — total 720 ms; IMPACTO en el frame 3 (a los 370 ms).

frame	pose [visual]	ms	inicio	fase	lógica
0	ANTICIPATION	150	0	prep	
1	WIND UP	140	150	prep	
2	SWING	80	290	prep	
3	IMPACT	80	370	activo	ACTIVO, IMPACTO

frame	pose [visual]	ms	inicio	fase	lógica
4	FOLLOW THROUGH	110	450	rec	
5	RECOVERY	160	560	rec	

attack2 — total 875 ms; IMPACTO en el frame 3 (a los 450 ms).

frame	pose [visual]	ms	inicio	fase	lógica
0	ANTICIPATION	170	0	prep	
1	WIND UP	170	170	prep	
2	SWING	110	340	prep	
3	IMPACT	95	450	activo	ACTIVO, IMPACTO
4	FOLLOW THROUGH	140	545	rec	
5	RECOVERY	190	685	rec	

parry — total 560 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	GUARD UP	60	0	prep	
1	DEFLECT	70	60	activo	ACTIVO
2	SPARK	90	130	activo	ACTIVO
3	PUSH BACK	100	220	rec	
4	READY	110	320	rec	
5	RECOVERY	130	430	rec	

hit — total 630 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	IMPACT	70	0	activo	
1	RECOIL	90	70	rec	
2	STAGGER	110	160	rec	
3	STAGGER	110	270	rec	
4	RECOVER	120	380	rec	
5	READY	130	500	rec	

block — total 610 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	RAISE	70	0	prep	
1	HOLD	110	70	activo	ACTIVO
2	HOLD	110	180	activo	ACTIVO
3	IMPACT	120	290	activo	ACTIVO
4	HOLD	110	410	activo	ACTIVO
5	LOWER	90	520	rec	

dodge — total 550 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	CROUCH	70	0	prep	

frame	pose [visual]	ms	inicio	fase	lógica
1	PUSH OFF	80	70	prep	
2	DASH	90	150	activo	ACTIVO
3	DASH	90	240	activo	ACTIVO
4	LANDING	100	330	rec	
5	RECOVERY	120	430	rec	

death — total 1500 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	HIT	110	0	activo	
1	STAGGER	130	110	rec	
2	KNEES	160	240	rec	
3	COLLAPSE	180	400	rec	
4	DOWN	220	580	rec	
5	DOWN	700	800	rec	

deflected — total 710 ms; se reescala a 0.7 s (0.9 s tras ganar el intercambio).

frame	pose [visual]	ms	inicio	fase	lógica
0	CLASH	60	0	activo	
1	RECOIL	100	60	rec	
2	OFF BALANCE	170	160	rec	
3	OFF BALANCE	170	330	rec	
4	REGAIN	120	500	rec	
5	READY	90	620	rec	

counter — total 545 ms; IMPACTO en el frame 3 (a los 220 ms).

frame	pose [visual]	ms	inicio	fase	lógica
0	CATCH	70	0	prep	
1	TWIST	80	70	prep	
2	LUNGE	70	150	prep	
3	SLASH	75	220	activo	ACTIVO, IMPACTO
4	FOLLOW THROUGH	110	295	rec	
5	READY	140	405	rec	

heavy — total 1080 ms; IMPACTO en el frame 4 (a los 680 ms); HOLD (frame 2) es la retención del plan (0.35-0.55 s).

frame	pose [visual]	ms	inicio	fase	lógica
0	WIND UP	170	0	prep	
1	RAISE	180	170	prep	
2	HOLD	260	350	hold	
3	SLAM	70	610	prep	
4	IMPACT	130	680	activo	ACTIVO, IMPACTO
5	RECOVERY	270	810	rec	

16.4 Zombi (assets/fodder_zombie_atlas.json) — 1.05 × CHAR_H

attack: el plan estira los frames 0-2 a 0.8 s exactos (carga 0.62 + suelta 0.18); recuperación IMPACT→fin 1000 ms. Aturdido: frames 0, 1 y 2 de hit (80 ms cada uno) y se queda en el 2 (temblor **[visual]**). Sin run/parry/block/dodge.

Frame 118×102 px, pivote [55.86, 85.67999999999999] **[visual]**. Direcciones: S, SW, W, NW, N, NE, E, SE.

idle — total 1020 ms; bucle.

frame	pose [visual]	ms	inicio	fase	lógica
0	NEUTRAL	170	0	rec	
1	INHALE	170	170	rec	
2	PEAK	170	340	rec	
3	EXHALE	170	510	rec	
4	SWAY	170	680	rec	
5	RETURN	170	850	rec	

walk — total 900 ms; bucle.

frame	pose [visual]	ms	inicio	fase	lógica
0	CONTACT L	150	0	rec	
1	DOWN L	150	150	rec	
2	PASS L	150	300	rec	
3	CONTACT R	150	450	rec	
4	DOWN R	150	600	rec	
5	PASS R	150	750	rec	

attack — total 1800 ms; IMPACTO en el frame 3 (a los 800 ms).

frame	pose [visual]	ms	inicio	fase	lógica
0	ANTICIPATION	310	0	prep	
1	WIND UP	310	310	prep	
2	SWING	180	620	prep	
3	IMPACT	120	800	activo	ACTIVO, IMPACTO
4	FOLLOW THROUGH	330	920	rec	
5	RECOVERY	550	1250	rec	

hit — total 630 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	IMPACT	70	0	activo	
1	RECOIL	90	70	rec	
2	STAGGER	110	160	rec	
3	BRACE	110	270	rec	
4	RECOVER	120	380	rec	
5	READY	130	500	rec	

death — total 1500 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	HIT	110	0	activo	
1	STAGGER	130	110	rec	
2	KNEES	160	240	rec	
3	COLLAPSE	180	400	rec	
4	DOWN	220	580	rec	
5	STILL	700	800	rec	

16.5 Perro zombi (assets/fodder_dog_atlas.json) — 0.6 × CHAR_H

attack: 0.6 s exactos (carga 0.4 + suelta 0.2, salta en la suelta: la estocada empieza en la suelta, hasta 2 u); recuperación 900 ms. Sin run/parry/block/dodge.

Frame 102×77 px, pivote [47.879999999999995, 70.92] **[visual]**. Direcciones: S, SW, W, NW, N, NE, E, SE.

idle — total 900 ms; bucle.

frame	pose [visual]	ms	inicio	fase	lógica
0	NEUTRAL	150	0	rec	
1	INHALE	150	150	rec	
2	PEAK	150	300	rec	
3	EXHALE	150	450	rec	
4	SWAY	150	600	rec	
5	RETURN	150	750	rec	

walk — total 570 ms; bucle.

frame	pose [visual]	ms	inicio	fase	lógica
0	CONTACT A	95	0	rec	
1	WEIGHT A	95	95	rec	
2	PASS A	95	190	rec	
3	CONTACT B	95	285	rec	
4	WEIGHT B	95	380	rec	
5	PASS B	95	475	rec	

attack — total 1500 ms; IMPACTO en el frame 3 (a los 600 ms).

frame	pose [visual]	ms	inicio	fase	lógica
0	CROUCH	210	0	prep	
1	WIND UP	210	210	prep	
2	LUNGE	180	420	prep	
3	BITE	100	600	activo	ACTIVO, IMPACTO
4	LAND	300	700	rec	
5	RECOVERY	500	1000	rec	

hit — total 570 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	IMPACT	60	0	activo	
1	RECOIL	80	60	rec	
2	STAGGER	100	140	rec	
3	BRACE	100	240	rec	
4	RECOVER	110	340	rec	
5	READY	120	450	rec	

death — total 1440 ms.

frame	pose [visual]	ms	inicio	fase	lógica
0	HIT	100	0	activo	
1	STAGGER	120	100	rec	
2	BUCKLE	150	220	rec	
3	COLLAPSE	170	370	rec	
4	DOWN	200	540	rec	
5	STILL	700	740	rec	

17. Diferencias entre la documentación antigua y el código

Revisión hecha leyendo el código actual frente a los comentarios de cabecera de cada archivo y PROGRESS.md. En todos los casos **manda el código** (y es lo que describe esta especificación).

#	Dónde lo dice	Documentación	Código actual
1	player.js (W.PLAYER_PARAMS.walkSpeed), PROGRESS	marcha a 1.05 alturas/s (= 1.785 u/s)	con los pies anclados (activados por defecto) la marcha sale de los pies: 0.8-1.25 u/s según la dirección del sprite . walkSpeed y accel = 5 no se usan (la aceleración es «llegar a la marcha en 0.25 s»)
2	cabecera de controls.js	tocar al enemigo = atacar; mantener el dedo sobre él (o J) = golpe RETRASADO	tocar un enemigo que no es el objetivo lo selecciona ; tocar el objetivo = ligero; mantener ≥ 0.4 s = FUERTE con carga
3	cabecera de fighter.js	combo attack1 → attack2 → attack3 por la ventana IMPACT..RECOVERY; RETRASO manteniendo el ataque	el jugador usa la tabla L/H de moves.js (cola de 2, margen de 0.35 s tras el golpe). La cadena por next solo la usan los enemigos (eco). El RETRASO del jugador es código muerto: ningún control envía hold
4	cabeceras de fighter.js y combat.js, PROGRESS (Parry2)	el parry PERFECTO abre una ventana de contraataque indefendible de ~350 ms ($\times 1.6$ daño)	duelOnParry crea antes la ventana de riposte (t.rip) y counterT solo se fija si !t.rip; como t.rip no se borra nunca (solo al reaparecer), tras la primera ventana de riposte de la partida esos 350 ms no vuelven a abrirse : en la práctica lo sustituye el riposte (0.7 s / 0.32 s). counterT sí se usa tras el MIKIRI (0.35 s) y en el aire (airCounterT 0.6 s). Recomendación: en Unity, implementar el riposte y decidir explícitamente si se quiere también el contraataque de 350 ms

#	Dónde lo dice	Documentación	Código actual
5	cabecera de <code>automaton.js</code>	«los parries del jugador la llenan rápido (40-50 cada uno)»	18-28 por desvío (<code>attack1 18 / attack2 21</code> ; perfecto $\times 1.33$) +3 por cada desvío seguido en la misma cadena ; aturcido tras 5-6 desvíos de golpes sueltos (así lo comprueba <code>enemy.mjs</code>)
6	<code>W.AUTOMATON.prepK = 1.9</code> («preparación $\times 1.9$ »)	la preparación del autómeta es $\times 1.9$ más lenta	sin efecto: todos sus golpes llevan PLAN (carga/retención/suelta) y <code>startAttack</code> pone <code>slow = 1</code> cuando hay plan. Los tiempos salen de <code>wind/hold/rel</code>
7	cabecera de <code>combat.js</code>	hitstop de 60-120 ms	60-200 ms (perfecto pesado y mikiri 180, remate 160, deathblow del duelo 200)
8	PROGRESS / cabeceras	PERFECTO = últimos ~70 ms	$\text{early} \leq \min(70 \text{ ms}, \text{ventana}/2)$ (con spam la ventana encoge y el perfecto también) y se acepta hasta 4 ms después del impacto
9	<code>W.COMBAT.attackCost = [6, 7, 12]</code>	coste de stamina de <code>attack1-3</code>	para el jugador manda la tabla de <code>moves.js</code> (L 6, LL 7, LLL 12, H 18...); <code>attackCost</code> solo lo usan los enemigos sin tabla (eco) y el riposte ($\times 0.5$)
10	<code>W.COMBAT.holdMax = 0.6</code>	retraso máximo manteniendo el ataque	solo aplica al RETRASO (código muerto en el jugador, #3)

Notas que no son errores pero conviene saber: - La calibración positiva también **retrasa la resolución** de los golpes enemigos `calib` ms (para que la pulsación tardía cuente). - La guardia del jugador cubre 360°; el arco del golpe es un semiángulo ($\text{spin} \pm 180^\circ = \text{círculo completo}$). - El relleno recibe siempre 10 de daño por golpe del jugador (aunque sea un fuerte), por diseño.